App Development Project Report

App Dev Project Report

1. Student Details

Name: Manas Bhaskar Chaudhari
Roll Number: 23f2001944
Email: 23f2001944@ds.study.iitm.ac.in
About Me: I am a student pursuing my degree with an interest in full-stack web development, AIML and DS domains. I enjoyed working with Flask and Vue.js to build this Modern Application Development 2 Project.

---

2. Project Details

Project Title: Trekking Management Application (TMA)

Problem Statement:

Adventure and trekking organizations often rely on spreadsheets, phone calls, and manual coordination to manage treks, staff assignments, and participant bookings. This leads to inefficiencies such as overbooking, poor visibility into slot availability, and no centralized tracking of booking history. TMA is a web-based application that streamlines these processes by allowing Admins, Trek Staff, and Trekkers (Users) to interact with the system based on clearly defined roles.

Approach:

The application was built using Flask as the backend REST API framework, with SQLAlchemy as the ORM over a SQLite database. Role-based access control was implemented using Flask-JWT-Extended, with a single unified `User` model differentiating Admin, Staff, and Trekker roles. The frontend was built as a fully decoupled Single Page Application using Vue 3 and Bootstrap 5, communicating with the backend via Axios. Redis was integrated with Flask-Caching to cache frequently accessed trek listings, and Celery (with Redis as broker/backend) was used to handle background jobs — daily trek reminders, monthly admin reports, and asynchronous CSV export of booking history. Celery Beat was configured to schedule the recurring reminder and report jobs.

---

3. AI/LLM Declaration

I used Claude AI (Anthropic) to assist with building and debugging. The extent of AI/LLM usage is approximately 35-40%, concentrated mainly on frontend component scaffolding and Celery/Redis configuration. All core backend logic, database models, API design, authentication/authorization, and business rules (booking validation, slot management, role restrictions) were implemented and reviewed manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Flask-JWT-Extended	Token-based authentication and role-based access control
Werkzeug	Password hashing (security)
Celery	Background/async task processing
Flask-Login	User authentication and session management
Redis	Message broker and result backend for Celery, and cache store
SQLite	Lightweight local database for storing user data

---

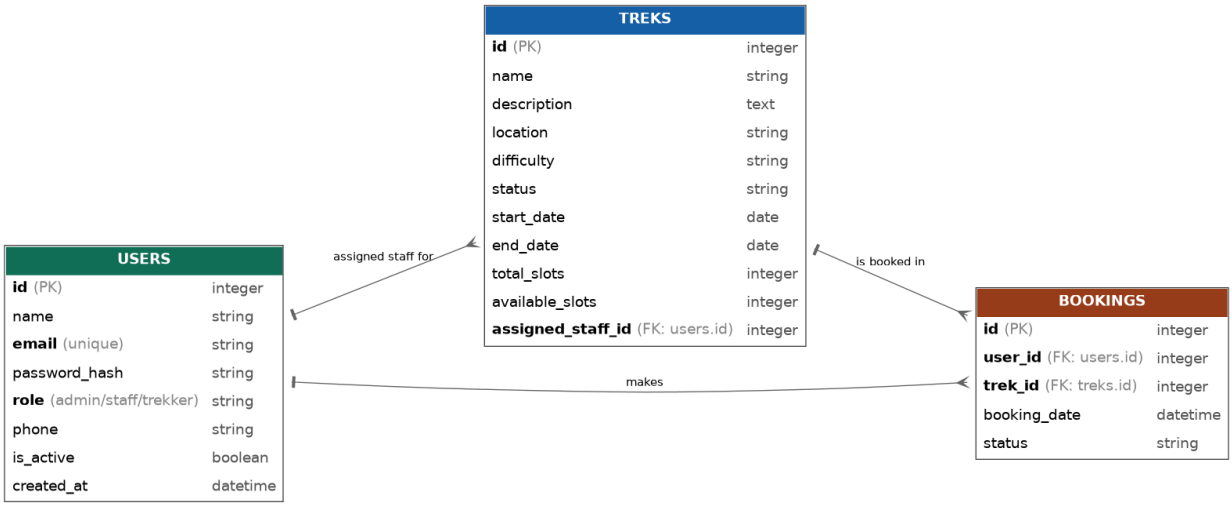
5. Database Schema / ER Diagram

Tables:

1. **User** — stores unified user data for all roles (id, name, email, password_hash, role, phone, is_active, created_at)
2. **Trek** — stores trek details (id, name, description, location, difficulty, status, start_date, end_date, total_slots, available_slots, assigned_staff_id)
3. **Booking** — records a user's trek booking (id, user_id, trek_id, booking_date, status)

Relationships:

- One-to-Many → User (Staff) → Trek (via `assigned_staff_id`)
- One-to-Many → User (Trekker) → Booking (via `user_id`)
- One-to-Many → Trek → Booking (via `trek_id`)



---

6. API Resource Endpoints

Endpoint	Method	Description
/api/register	POST	Register a new trekker account
/api/login	POST	Authenticate user and issue JWT access token
/api/admin/create-staff	POST	Admin creates a new trek staff account
/api/admin/users	GET	Admin fetches users/staff (filterable by role, search)
/api/admin/user/<id>/status	PUT	Admin activates/blacklists a user or staff member
/api/dashboard/stats	GET	Role-based dashboard statistics (admin/staff)

/api/treks	GET	Fetch treks (role-filtered, cached via Redis)
/api/create/trek	POST	Admin creates a new trek
/api/update/trek/<id>	PUT	Admin/assigned staff updates trek details
/api/delete/trek/<id>	DELETE	Admin deletes a trek (only if no bookings exist)
/api/book/trek/<id>	POST	Trekker books a trek
/api/bookings	GET	Fetch bookings (role-based: own/assigned/all)
/api/cancel/booking/<id>	PUT	Trekker cancels their booking
/api/user/profile	GET / PUT	View/update logged-in user's profile
/api/export/history	POST	Trigger async Celery job to export booking history as CSV
/api/export/status/<task_id>	GET	Poll status of the export job
/api/export/download/<filename>	GET	Download the generated CSV export

---

7. Video Presentation

Drive Link:
MAD2-VideoDemo.mp4